Dynamic Speculators

Project Working Record [1]

Abstract

This document records ongoing experiments with a conditional generator that produces LoRA factors for an EAGLE3 drafter from sets of target hidden-state summaries. It details the set encoder, shared decoder, and full-factor output heads, and records fixed-update data scaling and the decision to retire LoRA-bank warmup. It describes the training and serving interfaces, a supplied four-model throughput comparison on in-distribution and out-of-distribution workloads, activation-cache measurements, and an unresolved output-repeatability investigation. Completed shared-adapter concurrency measurements, a partial matched-budget conditioning-size study, and per-request continuous-batching results distinguish acceptance gains from serving throughput. A screenshot-based rank study records adapter variation, category separation, acceptance and token rate. Bounded runtime profiling is implemented, with GPU diagnostic evidence still pending. Every experiment states its available evidence, execution status, and missing information. The document is extended on request and does not present final conclusions.

1. Research Objectives and Questions

We investigate a generator that maps a set of prompt summaries to one shared adapter for a frozen EAGLE3 drafter. Separate serving studies compare the original drafter, trained static LoRA, shared-adapter generators and per-request generators. Checkpoint labels and protocol changes are retained within each study rather than pooled. Historical training ablations are recorded separately: fixed-update data scaling and the decision to discontinue bank-based generator warmup.

The working questions are:

1. How sensitive is generator validation behavior to training-pool size under a fixed optimizer-update budget?

2. How do generator checkpoints behave relative to the original drafter and a static trained adapter under a specified serving workload?

3. How do recorded measurements differ between Nemotron validation prompts and selected Red Hat benchmark categories?

4. What changes when one generated adapter is shared by a fixed batch of requests at different concurrencies?

5. Does conditioning on more examples improve a shared adapter when training data and optimizer updates are matched?

6. Do per-request adapters preserve acceptance while permitting native continuous admission, and does better acceptance yield higher throughput?

7. How does LoRA rank relate to adapter variation, category separation, acceptance and serving token rate?

8. What time and memory costs arise from obtaining and reusing target activations during training?

These questions remain open. A numerical table records an observation under its measurement conditions; it does not establish general superiority, statistical significance, lossless acceleration, or completion of a planned experiment.

1.1. Record Conventions

Experiment identifiers remain stable as evidence is appended. We use *planned*, *implemented*, *partially measured*, *measured*, and *investigation deferred* as execution statuses. Evidence quality is stated separately: a measured value may still have incomplete provenance or no repetition. Dates distinguish receipt of evidence from execution when the latter is unknown. This version includes evidence available through 2026-10-25.

[1] Internal experimental record, version 04. Correspondence to: Project record <repository issues>.

Preprint. October 25, 2026.

2. Method and System Description

2.1. Set-Conditioned Adapter Generation

The generator receives one summary per conditioning prompt, together with its prompt length. Current serving uses the hidden states at the last prompt token from the configured target layers. Prompt conditioning excludes response tokens. For a conditioning set $S = \{(h_i, \ell_i)\}_{i=1}^{n}$, a shared encoder produces representations z_i. The condition is constructed from their mean, population standard deviation, and the logarithm of the set size:

$$c = F([\mathrm{mean}_i\, z_i,\ \mathrm{std}_i\, z_i,\ \log(1+n)])\,. \tag{1}$$

Module and layer descriptors are combined with c before decoding full LoRA factors. For each adapted projection p, the resulting weight is

$$W_p' = W_p + \frac{\alpha}{r} B_p A_p. \tag{2}$$

The current implementation adapts the drafter's V and output projections. The set produces one adapter shared by its examples, rather than independent adapters for each example. Set conditioning therefore differs from the number of independently scheduled inference requests.

2.2. Generator Architecture

The generator is a full-factor hypernetwork: its outputs are the adapter's actual matrices, rather than indices into a stored adapter bank. For a prompt summary of dimension d_h, the shared example encoder takes $[h_i, \log(1+\ell_i)]$ through a $(d_h+1)\to 512$ linear layer, GELU, a $512\to 128$ layer, and LayerNorm. Pooling the 128-dimensional mean and population standard deviation with $\log(1+n)$ gives 257 features. A $257 \to 128 \to 64$ MLP with GELU produces the condition. Pooling makes the result invariant to the ordering of conditioning examples; the count feature distinguishes different set sizes.

Each projection has a learned 32-dimensional module descriptor, alongside a 32-dimensional layer descriptor. In the current single-layer drafter the layer embedding has one entry. Concatenating both with the condition gives 128 features. A shared linear layer and GELU expand these to width 512, followed by two residual MLP blocks. Each block applies LayerNorm, linear, GELU, and linear before adding its input back.

Separate linear output heads for each projection produce $A_p \in \mathbb{R}^{r \times d_{\mathrm{in},p}}$ and $B_p \in \mathbb{R}^{d_{\mathrm{out},p} \times r}$. The rank-32 generator used in the recorded logs has approximately 293 million parameters; the large full-factor heads dominate its size. The bottleneck condition is shared, while the factors depend on the projection descriptor. Initially the head weights are zero, the A biases receive Kaiming initialization, and the B biases are zero, so the initial adapter update is zero.

This structure gives one representative LoRA for a conditioning set. An ordinary LoRA baseline instead learns one persistent pair of factors per projection. The generator's larger trainable capacity is a confound when comparing these approaches; matching rank alone does not match trainable parameter count. Architecture dimensions above follow the inspected implementation defaults; per-run configuration artifacts are needed to confirm historical overrides.

2.3. Training Interface

The target and base drafter remain frozen while generator parameters are optimized through generated factors. The current recipe uses `lk_hybrid`, rank 32, and alpha 64. Conditioning-set sizes are sampled from 1–8; four episodes are accumulated per optimizer update in the documented recipe. Token-budget microbatches share the episode's factors. Context-4 validation is a separate evaluation setting. Gradients from the drafter loss flow through the generated A/B factors into the encoder, decoder, and output heads. Thus the generator learns adapters that serve the drafting objective directly, without requiring a previously trained adapter as a target.

Training replays saved prompt/response token trajectories through an external vLLM activation extractor. Only activations needed by the upcoming work are constructed, and CPU RAM retains reusable tensors within a bounded cache. The training pipeline does not require downloading the full activation dump. These describe the implementation and recipe, not verified configurations of every historical measurement.

2.4. Serial and Fixed-Batch Serving

Serial serving conditions one adapter on one prompt. The new fixed-batch mode collects a closed group, obtains all prompt summaries from full target prefill, generates one adapter, and merges it into the shared drafter before draft prefill. The weights remain fixed until all group members finish or are aborted. New requests cannot replace members that finish early. A subsequent group receives its own adapter after the previous group ends.

The configured concurrency is a capacity, not a guarantee of actual batch size. A collection timeout permits partial groups, including the final workload tail. Original-drafter and static-LoRA baselines use the same closed admission policy in the new sweep. Static LoRA uses a permanent merge. No GPU throughput or acceptance result for this batching policy is included in this version.

3. Shared Experimental Setup

3.1. Models and Data

The documented model family is Qwen3-8B with the Red Hat EAGLE3 drafter. The historical shared-cohort suite compares the original drafter, a trained ordinary LoRA, and generator snapshots labeled 2,000 and 4,000 steps. Generator data size is reported as 24K examples. Exact training progress must ultimately come from checkpoint metadata; labels are retained without treating them as verification.

Nemotron math, code, STEM, and chat are designated in-distribution evaluation categories. Red Hat HumanEval, math reasoning, and summarization are designated out-of-distribution categories in this comparison. These labels describe the chosen evaluation sources; they do not by themselves establish absence of all semantic overlap with training. Validation rows are excluded from optimization by the documented trajectory membership contract.

The full-data workflow schedules six training subsets per domain, totaling 24K examples, and supports ten planned epochs. It saves Hub snapshots every 1,000 optimizer updates and at completion, with context-4 validation every 500 updates in the current recipe. The schedule is not evidence that ten epochs have completed. This report includes only the supplied checkpoint observations.

3.2. Metrics and Measurement Boundaries

We keep the following quantities distinct:

- **Training and validation losses:** optimization/evaluation values under a specified conditioning and microbatch policy.
- **Offline agreement:** drafter/target argmax agreement in offline evaluation, not measured speculative acceptance.
- **Serving acceptance:** native speculative-decoding counters during actual generation. For BATCH-001/003, acceptance length is $L = 1 + A/D$, where A counts accepted draft tokens and D draft cycles. Acceptance rate is A/P for P proposed draft tokens; with three proposals per cycle, $L = 1 + 3A/P$. Neither quantity measures output accuracy.
- **Throughput:** aggregate generated output tokens divided by measured wall seconds, including prefill and serving overhead.
- **Latency and systems timings:** request latency, first-token time, and stage wall time, when their artifacts are available.

3.3. Recipe Versus Verified Run Metadata

The independent serving implementation fixes BF16, eager execution, the V1 runner, and a pinned vLLM 0.31.0 integration. Suite defaults specify a seed, sample limit, output budget, and warmup count. These defaults must not be substituted for missing metadata from the supplied throughput summary. The later BATCH-001/003 summaries establish 50 successful requests per measurement and matched category workload hashes. They still omit full hardware, launch/source and checkpoint provenance. BATCH-002 supplies only a throughput table, and GEN-005 supplies screenshots with unknown workload/runtime settings. The cache pilot separately reports an A100 80 GB. None establishes repeated-run uncertainty. Native continuous admission in BATCH-003 must not be conflated with the fixed-cohort policy used for every BATCH-001 arm.

4. Experiments and Recorded Measurements

4.1. GEN-001: Data Scaling at Fixed Optimizer Budget

Status: Preliminary; single-seed visual evidence.

Evidence: W&B screenshots, including the comparison with ordinary LoRA supplied 2026-10-23. Raw metric exports and immutable run configurations are unavailable.

Question. Does more training data improve validation agreement within the same number of optimizer updates?

Setup. The direct generator arms use nested pools of 4K, 8K, and 16K examples: one, two, or four subsets per domain across math, code, STEM, and chat. All three run for 1,500 updates with seed 42. The documented recipe uses `lk_hybrid`, four episodes per update, training context sizes 1–8, rank 32, alpha 64, and an 8,192-token microbatch budget. AdamW uses learning rate 10^{-4}, weight decay 0.01, gradient clipping 0.85, and 100 learning-rate warmup updates. This scheduler warmup is distinct from bank reconstruction. Validation uses 100 held-out rows per domain from subset 00000, grouped at context size 4, every 250 updates including updates 0 and 1,500. The additional curve is labeled `scale-1-lora-gseed42`; its intended comparison is ordinary LoRA trained on the 4K pool at the same update budget. Historical configuration artifacts are needed to verify all matched settings.

Observed behavior. In Figure 1, generator validation curves largely overlap across data scales. Most improvement occurs early, with smaller gains thereafter. The ordinary LoRA curve improves more slowly and finishes visibly below the generators on math, code, and STEM. Chat differences are smaller, with a late dip in the scale-1 generator curve. Training loss is noisy, while the generator trends finish below the ordinary LoRA trend.

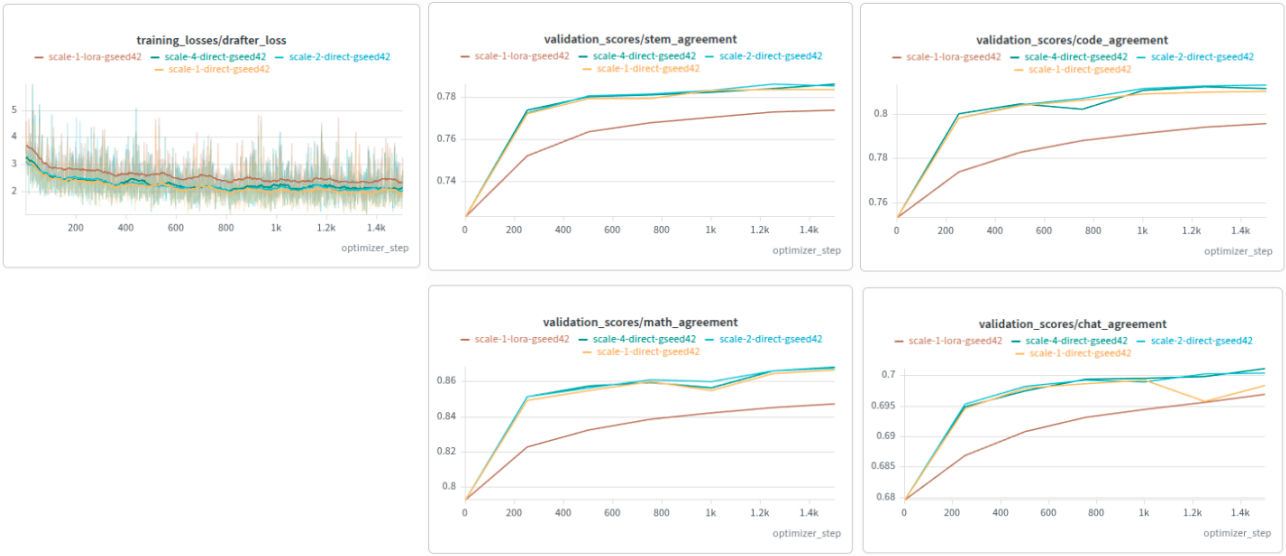


Figure 1. GEN-001: supplied training-loss and domain-validation plots. The three direct generator arms use 4K, 8K, and 16K training examples with the same 1,500-update budget. Ordinary scale-1 LoRA is included as a training baseline. Agreement is an offline validation metric, not measured serving acceptance. Curves are visual evidence; no raw values or uncertainty estimates are supplied.

Working interpretation. On this split, seed, and update budget, pool size has a small observed effect compared with the gain from training itself. Increasing available data fourfold does not produce a correspondingly large validation gain in these curves. This is a fixed-update comparison: it neither matches epochs nor guarantees equal processed tokens, compute, or wall time. Larger pools receive less average reuse. The result therefore motivates asking whether longer training or broader validation would reveal additional data benefits, rather than concluding that data quantity is generally unimportant.

Limits. Validation rows are held out, but their subset identity is represented in every training pool. One seed and screenshots cannot establish statistical equivalence or a precise ranking. Generator versus LoRA differences also include the much larger generator parameter count. These training curves do not add an older generator row to the current serving benchmark tables.

4.2. GEN-004: Bank Warmup and the Direct-Training Decision

Status: Preliminary adaptation observation; bank workflow retired.

Evidence: Adaptation screenshot supplied 2026-10-23; historical protocol recorded in the 2026-10-07 working notes. Exact execution date and raw exports unavailable.

Question. Does reconstructing previously trained bank LoRAs provide a useful lasting advantage over training a fresh generator on the original data?

Setup. The historical study adapts four initializations on `math-00001`: a bank-pretrained generator, a fresh generator, a transferred ordinary LoRA, and a fresh ordinary LoRA. The recorded protocol uses seed 42, last-token conditioning, training context sizes 1–8, and one adaptation epoch. The supplied plot spans roughly 200 optimizer updates and evaluates context 8; this is distinct from GEN-001's context-4 validation. Both generator arms are optimized through the drafter loss during adaptation.

Observed behavior. The pretrained generator starts at a lower validation loss than the fresh generator. By the end of the displayed adaptation, the two generator validation curves meet at a similar loss. The transferred ordinary LoRA changes little, while the fresh ordinary LoRA improves but remains above the generators. Figure 2 therefore shows an initialization benefit from bank warmup without an observed final-loss advantage at this budget.

Design decision. We retired the LoRA-bank approach as unhelpful for the current workflow. We use the original examples directly to train the generator with the drafting loss. Bank reconstruction first turns those examples into trained LoRA factors and then asks the generator to reproduce that proxy. Direct training optimizes the desired drafting objective itself and removes both bank construction and proxy reconstruction from the workflow. The observed fresh-generator catch-up supports choosing this simpler route; no further bank experiments are part of the current plan.

Scope of the claim. Direct training is the chosen, more useful approach for this project, rather than a demonstrated

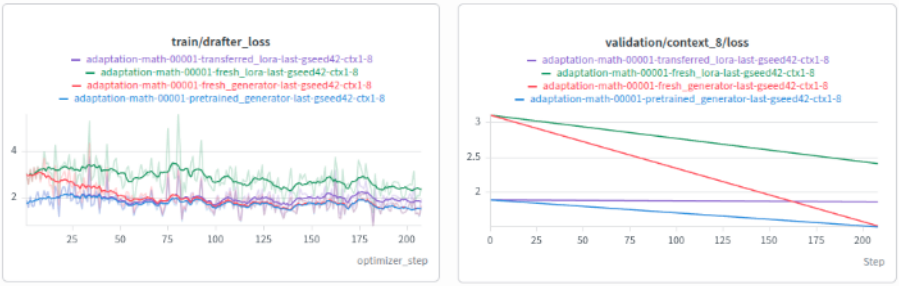


Figure 2. GEN-004: supplied adaptation curves for pretrained/fresh generators and transferred/fresh ordinary LoRAs. Bank pretraining improves the starting point, but the fresh generator reaches a similar final context-8 validation loss. The displayed comparison does not match total pretraining-plus-adaptation cost.

universal ordering of the methods. This adaptation study is not a matched-total-budget comparison of using the same examples for bank warmup versus direct pretraining. It also mixes conditioning and parameter-capacity differences between generators and ordinary LoRA. Raw metrics, repeated seeds, and complete historical provenance are missing. We retain the new screenshot without reusing approximate endpoint values from older screenshots in the working notes.

4.3. GEN-003: Current Four-Model Throughput Record

Status: Measured; supplied summary, incomplete provenance.

Evidence: Suite `comparison-v2`; received 2026-10-22. Execution date not supplied. Tables are generated from the frozen JSON transcription.

Question. What throughput values were recorded for the four model variants on the selected in-distribution and out-of-distribution categories?

Setup. The source is the suite owner's supplied `summary.md`. Tables 1 and 2 preserve all 28 reported measurements without adding older model rows. Step counts follow the supplied labels. Correctness was not checked. This record does not include the native per-request outputs, latency statistics, acceptance counters, or server provenance for this initial submission. The later BATCH-001 detailed summary adds acceptance, median latency, counters and workload hashes for these reused C1 values; these are additional artifacts, not new repeated runs.

Table 1. In-distribution Nemotron: recorded aggregate output tokens/s. Correctness was not checked; step counts follow supplied labels.

Model	Math	Code	STEM	Chat
EAGLE3 base	64.34	60.93	59.28	54.97
Trained LoRA	72.50	64.79	64.23	56.35
Generator, 2K label	77.58	68.11	66.50	57.58
Generator, 4K label	79.11	69.83	67.92	56.49

Table 2. Out-of-distribution Red Hat: recorded aggregate output tokens/s. Correctness was not checked; step counts follow supplied labels.

Model	HumanEval	Math reasoning	Summarization
EAGLE3 base	68.32	66.55	58.29
Trained LoRA	67.24	67.19	58.30
Generator, 2K label	69.24	67.99	58.81
Generator, 4K label	69.44	68.32	58.69

Record notes. The values are recorded separately by category and checkpoint. We do not pool categories into an overall score or infer a final model ranking. The checkpoint pair provides observations at two reported training stages, not a repeated or controlled causal estimate of additional training. The two tables do not establish output quality or a target-only decoding speedup.

Missing evidence. Native request-level GuideLLM results, effective configurations, hardware metadata, checkpoint hashes and repeated measurements remain unavailable. BATCH-001 supplies the later workload fingerprints.

4.4. SYS-001: Cold and Warm Activation Reuse

Status: Measured; console timings transcribed in the cache record.

Evidence: Pilot logs received 2026-10-16; reported A100 80 GB, 400 validation payloads evaluated in context-4 groups.

Question. What stage wall times were observed before and after activation payloads were available in the shared RAM cache?

Table 3. Validation wall seconds from the cache pilot. The first pass includes extraction and startup effects.

Category	First pass (s)	Warm pass (s)
Math	35.18	6.25
Code	24.49	5.47
STEM	15.53	3.41
Chat	21.70	4.93
All categories	96.90	20.06

Record notes. The payloads occupied 16.81 GiB. Subsequent jobs in the same process reported initial validation passes of 19.99 and 16.86 seconds. The first pass includes extraction and startup effects, whereas the warm pass still performs generator/drafter evaluation. The difference is an end-to-end observation, not an isolated cache microbenchmark or inference speedup. In a direct-training pilot the cache reached 63.97 GiB, with 365 evictions and 69.67 GiB final process RSS. Later prefetch and summary-cache changes have no separate GPU timing comparison in the evidence included here.

Missing evidence. Repeated cold/warm trials with separated extraction, construction, transfer, and compute timings remain available follow-up work.

4.5. GEN-002: Output Repeatability Investigation

Status: Investigation deferred; exact sequence comparisons recorded.

Evidence: Local correctness reports summarized in the Markdown registry, observation dated 2026-10-21. Raw reports are not archived with this report.

Question. Were target-only and speculative outputs repeatable under the supplied prompt/settings fingerprints?

On 50 Nemotron math prompts, original versus repeated target-only decoding matched exactly for 23/50 sequences; original versus repeated EAGLE3 decoding matched for 50/50. Repeated target-only versus repeated EAGLE3 matched for 17/50. These are whole-sequence match counts, not token-error rates.

Record notes. The cause remains unresolved. Runtime environment, numerical behavior, and decoding implementation differences have not been isolated. Current timing experiments explicitly record correctness as unchecked or unverified. No correctness claim is inferred from the throughput tables.

Missing evidence. Preserve and compare complete runtime provenance and repeat a controlled reference experiment before making an exact-output claim.

4.6. BATCH-001: Shared-Adapter Concurrency Sweep

Status: Measured summary supplied; preliminary, correctness unchecked.

Evidence: 112 measurements: four models, C1/C2/C4/C8, seven categories. Detailed summary received 2026-10-24; C1 values reuse GEN-003.

Question and protocol. Does one adapter conditioned on a group improve acceptance and serving throughput as concurrency increases? All four arms use fixed closed cohorts: vacant slots remain empty until the last request finishes. The generator produces one shared adapter per cohort and merges it into the frozen drafter, restoring the base afterward. Original EAGLE3 and static LoRA use the same admission restriction. This is a matched-policy comparison, not a benchmark of unrestricted native vLLM serving. Each measurement completes 50 requests; workload hashes match across all 16 variants per category.

Budget and identity limits. The static-LoRA recipe describes 4K examples and 1,500 updates, while the generator labels describe 24K training. G2 denotes `generator-24k-2000step`, mapped by the example configuration to `epoch-0002`; an earlier epoch-2 archive recorded 2,676 updates. G4 denotes `generator-4000step`. These labels do not independently verify exact training progress, hashes or matched compute. Native request-level records, hardware and launch/source provenance are unavailable.

Table 4. BATCH-001: unweighted means of per-category throughput changes for G4. These are not pooled-workload speedups.

C	ID/base	OOD/base	ID/LoRA	OOD/LoRA
1	+13.73%	+1.66%	+5.72%	+1.87%
2	+14.65%	+1.79%	+2.98%	+4.09%
4	+14.88%	-0.82%	+4.34%	+0.61%
8	+13.08%	+0.20%	+1.08%	-1.20%

Table 5. BATCH-001: ID acceptance length at C8, including the baseline token; higher is better.

Model / C	Math	Code	STEM	Chat
EAGLE3 / 8	2.419	2.305	2.233	2.087
Static LoRA / 8	2.810	2.520	2.471	2.162
G4 label / 8	2.944	2.618	2.537	2.198

Observed behavior. Both generators exceed original EAGLE3 throughput in every ID category at all four concurrencies. The G4 ID category-mean advantage over static LoRA narrows to 1.08% at C8, although its

acceptance-length gains there are 4.75/3.85/2.66/1.68% for math/code/STEM/chat. OOD behavior is mixed: at C8 G4 trails EAGLE3 by 4.68% on HumanEval, and trails static LoRA by 1.30% on math reasoning and 2.64% on summarization. An OOD mean near zero does not establish equivalence. Full ID/OOD throughput rows appear in the appendix.

Latency and execution counters. Recorded generator/merge/restoration GPU time accounts for only 0.070–0.105% of measurement wall time across the generator arms. It excludes uninstrumented CPU work and other synchronization. Adapter records contain 25/13/7 completed cohorts for 50 requests at C2/C4/C8, the minimum possible counts; this does not reveal vacant-slot time within each cohort. C8 mean cohort size is $50/7 = 7.143$.

Table 6. BATCH-001: median TTFT (ms) under closed cohorts. These measurements do not belong to BATCH-002 checkpoints.

Model / C	Math	Code	STEM	Chat
EAGLE3 / 1	43.5	48.1	43.6	43.9
Static LoRA / 1	45.0	50.8	45.0	44.9
G2 label / 1	47.6	53.7	48.4	47.8
G4 label / 1	46.7	53.0	47.3	49.6
EAGLE3 / 8	676.8	930.9	369.4	1584.7
Static LoRA / 8	373.0	662.5	407.8	1410.8
G2 label / 8	350.3	659.0	393.7	1382.6
G4 label / 8	399.3	713.2	392.7	1447.6

G4 chat median TTFT rises from 49.6 ms at C1 to 1,447.6 ms at C8 while aggregate throughput increases. The 2–3 ms generator invocation alone cannot account for that first-token delay. Queueing, cohort completion and prefill are not separated in the summary. HumanEval at C8 also illustrates why acceptance alone is insufficient: G4 acceptance length is 2.579 versus EAGLE3's 2.541, despite lower throughput.

Working assessment. Useful ID adaptation is observed under closed cohorts, but a strong incremental deployment benefit from our added sharing and admission policy is not established. Ordinary GPU batching itself remains useful: most C8/C1 scaling is also available to the original drafter. Comparing within one category and concurrency is essential. Reused C1 values and LoRA startup retries are not independent repeats or controls for runtime drift.

4.7. BATCH-002: Conditioning Size at Matched Data and Updates

Status: Partially measured; supplied C8 throughput only.

Evidence: Table received 2026-10-25 for static LoRA and separately trained K1/K2/K4 generators. K8 is unreported; native acceptance/latency unavailable.

Question and intended setup. Does conditioning on more prompt summaries improve a generator at a matched 24K-example, 1,000-update budget? Four separate generators are configured for $K = 1, 2, 4, 8$, alongside a static LoRA. Every episode uses eight loss examples and selects K of their summaries reproducibly; all eight still contribute loss. Four episodes per update yield 32,000 example presentations per arm, rather than 1,000 updates split between arms. Intended shared settings are seed 42, rank 32/alpha 64 V/O adapters, `lk_hybrid`, AdamW LR 10^{-4}, weight decay 0.01, clipping 0.85, 100-update LR warmup and an 8,192-token microbatch budget. Architecture and initial generator weights match. Effective run provenance is still missing.

Table 7. BATCH-002: supplied ID output tokens/s at C8; 24K examples and 1,000 updates per arm are intended matched budgets. K8, acceptance and TTFT have not been supplied.

Arm	Math	Code	STEM	Chat
Static LoRA	429.55	386.78	392.63	282.77
Generator K1	453.27	407.25	404.92	287.20
Generator K2	456.32	403.75	399.54	287.52
Generator K4	455.18	401.66	395.86	287.62

Serving holds concurrency at eight with one shared adapter per closed cohort, conditioned on $\min(K, \text{actual cohort size})$ summaries. Training K and serving K change together across separately trained models; this is not an inference-only ablation of one checkpoint. The intended export has 64 held-out prompts per category, 256-token caps and three warmups, but the supplied table alone cannot verify effective request counts or workload identities.

Observed behavior and limits. All three reported generators exceed matched-budget static LoRA on all four ID categories. K2 leads math, K1 leads code/STEM, and K4 leads chat. Larger K does not produce monotonically higher throughput. Missing K8, acceptance, actual conditioning counts and repeats prevent a complete ranking or a claim that larger conditioning sets are intrinsically unhelpful. Combined with BATCH-001, this supports the engineering assessment that the current fixed-cohort sharing technique has limited observed incremental benefit; it does not establish that GPU batching is ineffective.

4.8. BATCH-003: Per-Request Adapters with Continuous Admission

Status: Measured C1/C2/C4 summaries; correctness unchecked.

Evidence: 24 supplied measurements received 2026-10-25, two arms and four ID categories. C8 and complete runtime provenance are absent.

Question and protocol. Can each request retain its own generated LoRA while the native scheduler continuously admits and retires requests? The intended generator checkpoint is `step-00004000` of `azeeer/eagle3-generator-all24k-10ep-v1`. Each request conditions once on its own prompt summary. Draft-only native multi-LoRA executes request-specific V/O factors; no shared weight merge is performed. The intended baseline is original EAGLE3 with original vLLM sources and native continuous admission. BF16, eager V1 execution and other documented runtime restrictions still apply; this is not a fully default-tuned vLLM comparison. Supplied labels do not verify imported source hashes, exact weights or hardware.

All matched pairs share workload hashes and complete 50 requests. Generator, preparation and completion counters each equal 50, with one conditioning example and one request per adapter. Mean actual draft batch sizes are about 1.95–1.96 at C2 and 3.77–3.84 at C4. These counters support concurrent execution of the path but do not independently establish per-request semantic isolation.

Table 8. BATCH-003: aggregate ID output tokens/s, native continuous admission. Baseline source identity is intended but not independently verifiable from the supplied summary.

Model / C	Math	Code	STEM	Chat
Native base / 1	62.98	59.49	57.57	53.43
Per-request G4 / 1	67.02	60.24	57.92	49.56
Native base / 2	123.50	117.16	113.06	105.33
Per-request G4 / 2	130.23	114.72	111.68	95.65
Native base / 4	234.25	222.77	222.86	194.59
Per-request G4 / 4	256.09	221.21	221.03	185.33

Table 9. Acceptance rate (percent). Native base and per-request G4 are BATCH-003; old shared G4 is BATCH-001. Old-shared rates use (length minus 1)/3. Cross-study rows are descriptive, not a controlled scheduling or adapter-policy ablation.

Model / C	Math	Code	STEM	Chat
Native base / 1	47.42	43.28	40.85	36.07
Per-request G4 / 1	65.17	54.24	51.03	39.63
Old shared G4 / 1	65.34	54.16	51.03	39.65
Native base / 2	47.33	43.39	40.62	36.08
Per-request G4 / 2	65.24	54.08	50.63	39.76
Old shared G4 / 2	64.47	54.72	50.59	39.98
Native base / 4	47.21	43.30	41.00	35.26
Per-request G4 / 4	65.13	54.13	50.73	39.33
Old shared G4 / 4	64.69	54.54	50.84	39.70

Acceptance versus throughput. Per-request acceptance exceeds native EAGLE3 throughout the reported ID grid. At C4, acceptance-length gains are 22.24/14.14/13.09/5.93% for math/code/STEM/chat. Throughput improves 5–9% on math, stays close on code/STEM and regresses 5–9% on chat. The proposal-quality hypothesis holds descriptively; broad serving acceleration does not. Per-request and earlier shared-generator acceptance rates differ by less than 0.8 percentage points in every matched category/concurrency cell. That is a cross-study observation, not evidence of equivalence: scheduling, adapter execution, conditioning policy and unverified runtime conditions differ.

Table 10. BATCH-003: median TTFT (ms), measured by the client; lower is better.

Model / C	Math	Code	STEM	Chat
Native base / 1	44.18	48.82	45.07	44.59
Per-request G4 / 1	55.07	60.47	55.08	55.36
Native base / 2	76.56	80.84	76.90	77.96
Per-request G4 / 2	93.56	98.86	93.81	93.89
Native base / 4	77.28	89.72	78.02	79.89
Per-request G4 / 4	92.46	105.26	92.38	93.05

Systems diagnosis still open. Per-request median TTFT is higher than native EAGLE3 by roughly 10–18 ms. One-time generator work costs 3.12–3.30 ms per request and loading costs 1.07–1.13 ms; their recorded sum is only 0.098–0.419% of category wall time. Recurring mapping and unmerged projection work are plausible costs, not yet measured causes. Although workload hashes also match BATCH-001 and C4 TTFT is much lower than the old closed-cohort G4 values, the change cannot be assigned solely to continuous admission. Some output-token totals differ slightly; matching prompts do not establish equal outputs. Retain native continuous admission while profiling adapter execution separately. No task-accuracy or verified-losslessness claim follows.

4.9. GEN-005: Rank, Adapter Variation and Serving Metrics

Status: Preliminary; three screenshot transcriptions.

Evidence: Received 2026-10-25. Geometry summary labelled “modules averaged”; acceptance and tokens/s plots include a displayed base-model reference.

Question and evidence boundary. Does increasing LoRA rank accompany more input-dependent adapter variation and better serving behavior? The supplied series covers ranks 8/16/32/64. Training budget, seeds, checkpoint identities, evaluation data, concurrency, hardware and exact metric formulas are unknown. The geometry and serving figures are recorded as one supplied study, but their checkpoint linkage is not independently verified. “Base model” is the figure’s label, not an established target-only or EAGLE3 identity.

Table 11. GEN-005: reported module-averaged adapter geometry. Gap is within-category minus between-category cosine similarity.

Rank	Dynamism	Within cos.	Between cos.	Gap
8	0.3424	0.9961	0.9586	0.0375
16	0.3921	0.9941	0.9102	0.0839
32	0.4639	0.9850	0.8521	0.1329
64	0.5158	0.9717	0.8145	0.1572

Table 12. GEN-005: reported dynamism by category. Definitions and normalization of these metrics are unavailable.

Rank	Chat	Code	Math	Reading
8	0.3688	0.2932	0.5191	0.1884
16	0.4398	0.3350	0.6110	0.1827
32	0.4986	0.4208	0.6879	0.2485
64	0.5191	0.4652	0.7387	0.3402

Table 13. GEN-005: additional analysis labels preserved from the supplied summary. eff_dim and erank are distinct metrics; their definitions and classifier protocol remain unavailable.

Rank	eff_dim	erank	cat_acc	max_cdist
8	1.01	7.11	73.5%	0.8866
16	1.02	12.04	80.5%	1.0687
32	1.13	18.02	90.0%	1.1781
64	1.30	24.26	90.5%	1.2255

Table 14. GEN-005: screenshot-transcribed serving metrics. Gains use the displayed base model; its identity and metric aggregation are unknown.

Rank	Accept.	Gain (pp)	Tok/s	Gain
Base	53.53%	–	36.1	–
8	59.63%	+6.10	39.8	+10.25%
16	60.33%	+6.80	40.3	+11.63%
32	60.92%	+7.39	40.6	+12.47%
64	61.40%	+7.87	41.0	+13.57%

Observed geometry. Overall dynamism increases from 0.3424 to 0.5158. Chat/code/math increase monotonically; reading dips at rank 16 before increasing. Math has the largest reported dynamism at every rank. Cosine similarity *decreases* both within and between categories, with a larger fall between categories; the within-minus-between gap grows from 0.0375 to 0.1572. Within- category similarity remains high. This is consistent with stronger category separation. Increasing `cat_acc` and `max_cdist` agree with that interpretation, subject to unknown definitions and classifier protocol. The reported `eff_dim` and `erank` are retained as distinct labels, without assigning unprovided formulas or units.

Observed serving behavior. All ranks exceed the displayed 53.53% acceptance and 36.1 tokens/s reference. Rank 8 to 64 adds 1.77 acceptance percentage points and 1.2 tokens/s (3.02% relative to rank 8). Successive acceptance increments are 0.70/0.59/0.48 pp, suggesting diminishing increments in this series. Small adjacent gaps have no repeat uncertainty. The original plots have interrupted vertical axes; numeric tables preserve absolute values.

Interpretation and next evidence. Higher rank accompanies more varied, more category-separated adapters and higher reported acceptance/token rate. This association does not establish that adapter dynamism causes the serving improvement. Unknown acceptance denominators and throughput aggregation prevent pooling these figures with BATCH-001/003. A matched static-LoRA rank sweep, resource costs, original metric script and checkpoint-linked repeated runs would be needed to isolate conditional benefits or choose an optimal rank.

5. Implementation and Investigation Notes

5.1. Activation Supply and Cache Lifetime

Training consumes local saved trajectories, with activations constructed on demand by an external vLLM extraction server. A bounded CPU cache retains full tensors, while prompt summaries can be reused separately. Cache capacity is not a cap on process RSS: temporary construction buffers, model state, and other allocations contribute additional memory. Persistent summaries, prefetching, and concurrency controls are implementation features; only the pilot timings in SYS-001 are treated as measurements here.

5.2. Checkpoint Publication and Recovery

The full-data training workflow supports local recovery and Hub snapshots containing generator weights, optimizer state, progress, RNG state, resolved configuration, and available provenance. Inference bundles can be downloaded without optimizer checkpoints. Recovery capability is recorded as workflow support; it does not demonstrate completion of the ten-epoch training plan.

5.3. Independent Serving and Benchmarking

The suite separates model serving from the GuideLLM client. It launches and cleans up owned server/client process groups, exports category workloads once, records attempt state, and can skip completed measurements with matching artifact checksums. Model type selects an original drafter, static-LoRA bundle, or generator bundle. Throughput can be measured without an additional correctness-generation pass; the absence of that pass is recorded explicitly.

5.4. Fixed-Batch Integration Boundaries

The new scheduler leaves vacant slots empty until a group finishes. It permits a partial group after a 50 ms collection window in the example configuration. The runtime rejects KV preemption or incomplete initial group admission rather than reuse draft cache entries under a different generated adapter. Larger concurrency also increases the prefill/profile token budget and may change GPU memory requirements. These constraints define BATCH-001 and the shared-adapter BATCH-002 protocol. GPU summary measurements are now recorded, while detailed scheduling traces remain unavailable.

Collection, queueing, uneven request completion, and partial tails are part of the measured serving protocol. Closed groups are not equivalent to unrestricted continuous batching. Batch membership may vary with arrival/completion order, even when the same prompt list is reused across model variants.

5.5. Per-Request Multi-LoRA and Native Baseline

The per-request controller generates factors once from complete target prefill, retains a request-specific CPU copy, and activates only scheduled request IDs in the draft-only native LoRA manager. Token mappings follow actual query offsets on every draft forward, including speculative rejection and later one-token steps. Request completion releases that request's adapter; paused request factors remain available. The native scheduler can refill free slots without changing other requests' adapters. CPU tests cover lifecycle and mapping invariants; aggregate BATCH-003 counters are not a full isolation or correctness test.

Baseline launches can create an isolated package view containing the original, checksum-verified copies of the three integration-patched vLLM files. The global installation is left intact, and worker status can report imported source hashes. This supports original-source EAGLE3 with native admission. Such source hashes were not included in the supplied BATCH-003 upload, so this capability must not be presented as independently verified provenance for those measurements.

5.6. Bounded Profiling: Implemented, Measurements Pending

Revision `dc872c5` adds a two-phase profiling command. An unprofiled pass provides ordinary timing; fresh servers then collect bounded native CPU/CUDA traces after warmup on the same frozen models and workload. The serial example compares original EAGLE3, singleton-conditioned merged adapters and singleton native multi-LoRA; the continuous example compares baseline and per-request adapters at C4. Example captures use math/chat, 16 prompts per category, 128-token caps and at most 30 worker iterations.

Custom ranges cover generation, packing/loading, mapping and draft/projection execution. Analysis records CPU inclusive/self time, GPU kernels and transfers, union activity and gaps, and captured request occupancy. Projection ranges include base GEMM work; isolated LoRA kernels require their own kernel rows. The native execute-model scope includes target work and preparation rather than providing an isolated verification timer. Unattributed activity remains unknown. A bounded trace may miss refills or completions, and whole-workload acceptance counters cannot supply captured-window accepted-token denominators.

CPU lifecycle tests, synthetic CUDA-event analysis, a real CPU profiler export and mocked HTTP/GuideLLM orchestration checks were performed locally. CUDA/vLLM profiling has not been executed locally and no user GPU traces have been supplied. Profiler timings are diagnostic and excluded from ordinary throughput tables. Thus recurring adapter kernels/routing remain hypotheses, not diagnosed causes.

6. Open Questions and Planned Evidence

This section is a work queue rather than a conclusion. Entries are retained or revised when new evidence arrives.

1. **Suite provenance.** Attach the native artifacts for `comparison-v2` and the per-request study; resolve exact checkpoint progress, hardware, source hashes and effective serving settings. Later aggregate summaries establish concurrency labels and prompt hashes, but do not replace full per-run provenance.

2. **Repetition.** Repeat matched serving measurements to record uncertainty rather than interpret small single-run differences. These are evidence requirements, not a new commitment to execute the repeated-run study previously declined by the owner.

3. **Adapter execution costs.** Run the prepared same-session C1 merged-versus-native-multi-LoRA control and C4 continuous profiling. Separate recurring LoRA kernels/mapping from one-time generation/loading. Record bounded trace coverage and avoid interpreting profiler throughput as production timing.

4. **Conditioning and rank.** BATCH-002 has no K8/acceptance/TTFT evidence; do not fill it from older checkpoints. GEN-005 needs metric definitions, matched configurations, checkpoint linkage and resource costs before causal or optimal-rank claims. Per-request and shared acceptance similarity alone does not establish equal conditional information value.

5. **Reference behavior.** Continue GEN-002 when requested, separating target-only repeatability from adapter-related behavior.

6. **Scaling interpretation.** Distinguish the fixed-update result from matched-epoch training and broader validation. GEN-001 has no repeated seeds or uncertainty estimates. Bank warmup is retired; GEN-004 documents the decision rather than scheduling further bank experiments.

7. **Training progression.** Extend verified checkpoint observations as snapshots become available. Do not infer later training outcomes from the planned epoch schedule.

8. **Systems costs.** Separate activation extraction, preprocessing, transfer, and training compute when a further timing study is commissioned.

Future report versions should add measurements under the relevant stable identifier, or create a new identifier if the comparison protocol changes. Missing measurements remain missing; implementation status is not substituted for an experimental result.

A. Evidence Registry and Reproducibility

A.1. Source Registry

Repository paths below identify the working records. The report's build manifest records hashes of the source files available when this PDF was built.

GEN-001 `docs/experiments/results/GEN-001-direct-data-scaling.md`; scaling screenshot copied unchanged into `reporting/assets/`.

GEN-004 `docs/experiments/results/GEN-004-bank-warmup-adaptation.md`; adaptation screenshot copied unchanged into `reporting/assets/`.

Architecture `src/speculators/generator/model.py` and `src/speculators/generator/config.py`, included in the source snapshot.

GEN-003 `docs/experiments/results/GEN-003-generator-suite-throughput.md`; the copied numeric source is `reporting/data/comparison-v2.json`.

SYS-001 `docs/experiments/eagle3_activation_ram_cache.md`; the copied timing source is `reporting/data/cache-pilot.json`.

GEN-002 `docs/experiments/results/GEN-002-http-token-equality.md`.

BATCH-001 `docs/experiments/results/BATCH-001-shared-adapter-concurrency.md`; frozen numeric sources are `reporting/data/shared-concurrency.json`, `shared-detail.json` and `shared-summary.json` in that data directory.

BATCH-002 `docs/experiments/eagle3_conditioning_24k_1k.md`; `reporting/data/conditioning-partial.json`.

BATCH-003 `docs/experiments/results/BATCH-003-per-request-continuous.md`; `reporting/data/per-request-summary.json` and `per-request-analysis.json`.

GEN-005 `docs/experiments/results/GEN-005-rank-dynamism-and-serving.md`; `reporting/data/rank-sweep.json`; all three supplied images are preserved unchanged in `reporting/assets/` and included in the source archive.

Serving/profiling `docs/experiments/eagle3_per_request_lora.md` and `docs/experiments/eagle3_runtime_profiling.md`; implementation revision `dc872c505499a50358d3a74e7c07ad68498adbac`, including source-only/CPU-checked profiling capabilities rather than supplied GPU diagnostic results.

Training `docs/experiments/eagle3_generator.md`, `docs/experiments/eagle3_generator_campaign_online.md`, and `docs/experiments/eagle3_generator_full_data.md`.

Datasets `docs/experiments/speculator_benchmarks_inventory.md`.

Historical training ablations appear separately from the four-model serving comparison. Screenshots preserve qualitative evidence without inventing raw metric exports. Bank reconstruction is retained as a historical decision record, not as an active experiment. The supplied adaptation image has a different endpoint scale from the older approximate transcription; numerical readings from these records are not pooled.

A.2. Reference Revisions and Artifact Requirements

The documented target reference is Qwen/Qwen3-8B at `b968826d9c46dd6066d109eabc6255188de91218`; the original drafter reference is RedHatAI/Qwen3-8B-speculator.eagle3 at `08610ffa01dd9f16731fe8f627b85905b6aa51c4`. The inspected Red Hat dataset revision is `2ae86affa2cb97a972b7fc681dd51c04fbff083e`. These reference revisions do not replace missing per-run checkpoint hashes.

Publishing an experiment requires its available training/evaluation/serving commands, resolved configurations, model hashes, package/runtime details, and source patches. Native GuideLLM outputs and workload manifests should accompany performance records. This report contains a summary-level transcription for GEN-003; it does not claim to include those missing run artifacts.

A.3. Commands and Manual Update Policy

The shared-batch experiment is launched after updating the pinned vLLM hook:

```
python \
  scripts/apply_vllm_generator_patch.py \
  --apply
BENCH_CONFIG=examples/eagle3-lora/\
benchmark_suite.yaml
speculators benchmark suite \
  --config "$BENCH_CONFIG" \
  --output-root output/shared-batch-v1
```

These are workflow commands, not a claim that this report's throughput tables were produced by that new batch configuration. Actual launch commands must be read from the associated run provenance. Use a persistent output directory on Colab. Every server launch must pass `--provenance-dir`; the suite supplies this automatically.

The prepared diagnostic workflows use separate output directories and the same automatic server provenance capture. They are commands to obtain future evidence, not the source of the measurements above:

```
PROFILE_CONFIG=examples/eagle3-lora/\
profile_generator_serial.yaml
speculators benchmark profile \
  --config "$PROFILE_CONFIG" \
  --output-root output/profile-serial-v1
PROFILE_CONFIG=examples/eagle3-lora/\
profile_generator_continuous.yaml
speculators benchmark profile \
  --config "$PROFILE_CONFIG" \
  --output-root output/profile-c4-v1
```

Append `--resume` to reuse valid completed artifacts. Unprofiled timing references and profiled diagnostics are stored separately; see the runtime profiling runbook for trace coverage and

analysis definitions.

Report updates occur only on explicit request. Markdown remains the working experiment record. Before extending a report, freeze new input measurements, update statuses and evidence dates, preserve earlier snapshots, and regenerate tables from their numeric sources. New compiler build timestamps do not change the evidence cutoff.

B. Report Change History

Version 01, 2026-10-22. Initial experimental record: current four-model in-/out-of-distribution tables, cache pilot timings, deferred repeatability investigation, and pending shared-batch GPU measurements. No final conclusions are asserted.

Version 02, 2026-10-23. Retitled *Dynamic Speculators*; expanded the generator architecture; added fixed-update data scaling with the ordinary LoRA baseline and the adaptation figure; recorded the decision to retire bank warmup in favor of direct drafting-loss training. Version 01 is preserved.

Version 03, 2026-10-23. Reduced Figure 2 to 70% of the text width for a more compact presentation of the supplied screenshot. Evidence unchanged; Version 02 is preserved.

Version 04, 2026-10-25. Replaces pending BATCH-001 performance status with the completed shared-cohort summary; adds matched-budget partial K results, per-request continuous-admission throughput/acceptance/TTFT and cross-study shared acceptance context. Adds screenshot-based rank/dynamism/category/serving tables. Records bounded profiling as implemented with GPU evidence pending. Numeric inputs, original rank images and current implementation/protocol sources are frozen with the new snapshot; Versions 01–03 remain preserved. Correctness and provenance limitations are retained without an overall model ranking.

C. Full Shared-Cohort Throughput Tables

These tables preserve the supplied BATCH-001 112-value sweep separately by in-distribution and out-of-distribution workload. They use the shared-cohort admission restriction for every arm and must not be read as native continuous baseline measurements. The C1 rows already appear in GEN-003 and are not new repeats. Checkpoint names G2/G4 preserve supplied labels, not verified progress.

Table 15. BATCH-001: fixed-cohort output tokens/s. C1 values are reused from GEN-003, not independent repeats. G2/G4 are labels.

Model / C	Math	Code	STEM	Chat
EAGLE3 / 1	64.34	60.93	59.28	54.97
EAGLE3 / 2	116.85	110.79	108.42	85.08
EAGLE3 / 4	213.73	201.16	202.13	157.48
EAGLE3 / 8	370.40	344.12	358.96	269.56
Static LoRA / 1	72.50	64.79	64.23	56.35
Static LoRA / 2	136.08	122.63	118.62	92.40
Static LoRA / 4	248.65	222.99	220.87	163.45
Static LoRA / 8	446.95	389.09	397.33	278.03
G2 label / 1	77.58	68.11	66.50	57.58
G2 label / 2	139.21	124.74	120.36	92.44
G2 label / 4	256.04	228.38	219.65	165.02
G2 label / 8	448.07	397.04	397.62	277.83
G4 label / 1	79.11	69.83	67.92	56.49
G4 label / 2	142.55	128.85	121.48	92.11
G4 label / 4	263.18	237.29	224.17	169.35
G4 label / 8	451.59	393.68	397.94	283.40

Table 16. BATCH-001: fixed-cohort output tokens/s. C1 values are reused from GEN-003, not independent repeats. G2/G4 are labels.

Model / C	HumanEval	Math reasoning	Summarization
EAGLE3 / 1	68.32	66.55	58.29
EAGLE3 / 2	121.51	124.04	108.60
EAGLE3 / 4	222.33	236.27	198.76
EAGLE3 / 8	386.17	419.55	327.71
Static LoRA / 1	67.24	67.19	58.30
Static LoRA / 2	116.13	122.64	107.49
Static LoRA / 4	215.16	235.21	197.72
Static LoRA / 8	366.84	433.09	348.00
G2 label / 1	69.24	67.99	58.81
G2 label / 2	118.98	124.27	107.19
G2 label / 4	217.22	239.08	194.58
G2 label / 8	362.63	422.07	342.32
G4 label / 1	69.44	68.32	58.69
G4 label / 2	122.91	128.52	109.25
G4 label / 4	220.45	234.68	196.91
G4 label / 8	368.11	427.44	338.82